Zárthelyi dolgozat

Bevezetés

A mellékelt projektet egészítse ki/javítsa ki a megadott instrukciók alapján. Az induló állapotban minden szükséges függőség, beállítás és üzenet megtalálható. Mivel ez nem egy design óra, ezért nem probléma, ha a felület nem ugyanúgy néz ki, mint a képeken. Ugyanakkor az sem hátrány, ha úgy készítik el, hogy hasonlítson, vagy esetleg szebb legyen. A dependency injection során, ahol lehetséges az interface-re hivatkozzon. A felületeken megjelenő szövegeket mindenhol I18n segítségével a messages resource bundle-ből rakja fel!

Ha nem fordul a kód, vagy nem indul el az alkalmazás, az automatikusan elégtelent jelent!

Feladatok

1. Ki-/bejelentkezés

A rendszer jelenleg nem használható, mivel nincs bejelentkező felület. Készítse el a LoginView osztályt. Ehhez a Vaadin beépített bejelentkező komponensét használja! Állítsa be, hogy bejelentkezés ezen a képernyőn keresztül történjen. A bejelentkezési oldal legyen a ‘/login’ útvonalon. Figyeljen arra, hogy hiba esetén kerüljön jelölésre, hogy hiba történt!

2. PetService

Implementálja a PetService interface-t. Az implementált metódusokban hívja meg a ‘PetRepository’ osztálynak a megfelelő metódusait. A ‘PetRepository’ osztályból két implementáció található a projektben. Egy ‘db’ és egy ‘mock’ nevű, ezek közül az utóbbit használja. A dependency injection során nem hivatkozhat közvetlenül az implementációra, csak annak a nevére és az interface-re. (Az importok között nem szerepelhet az InMemoryPetRepository.)

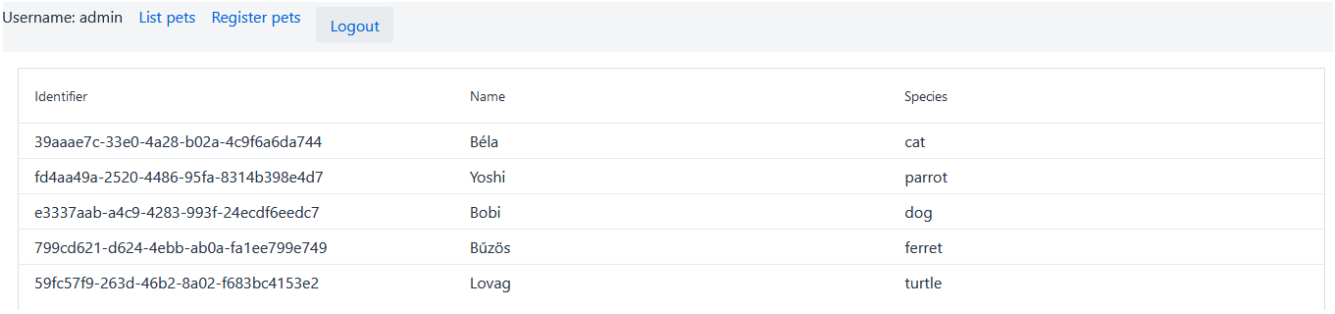
3. header

Implementálja a MainLayout osztályt. A fejlécben szerepeljen a bejelentkezett felhasználó neve, egy menü a 4-es és 5-ös feladatban létrehozandó képernyőkhöz, illetve egy kijelentkezés gomb. A menüpontokhoz célszerű a *RouterLink* komponenst használni.

4. Listázás

4 – 1 ListView

Készítse el a ListView képernyőt úgy, hogy a következőképpen nézzen ki:



A táblázatban lévő azonosító oszlop csak akkor látszódjon, ha ‘ADMIN’ jogosultságú felhasználó nyitja meg. (*SecurityService.getRolesAsString*)

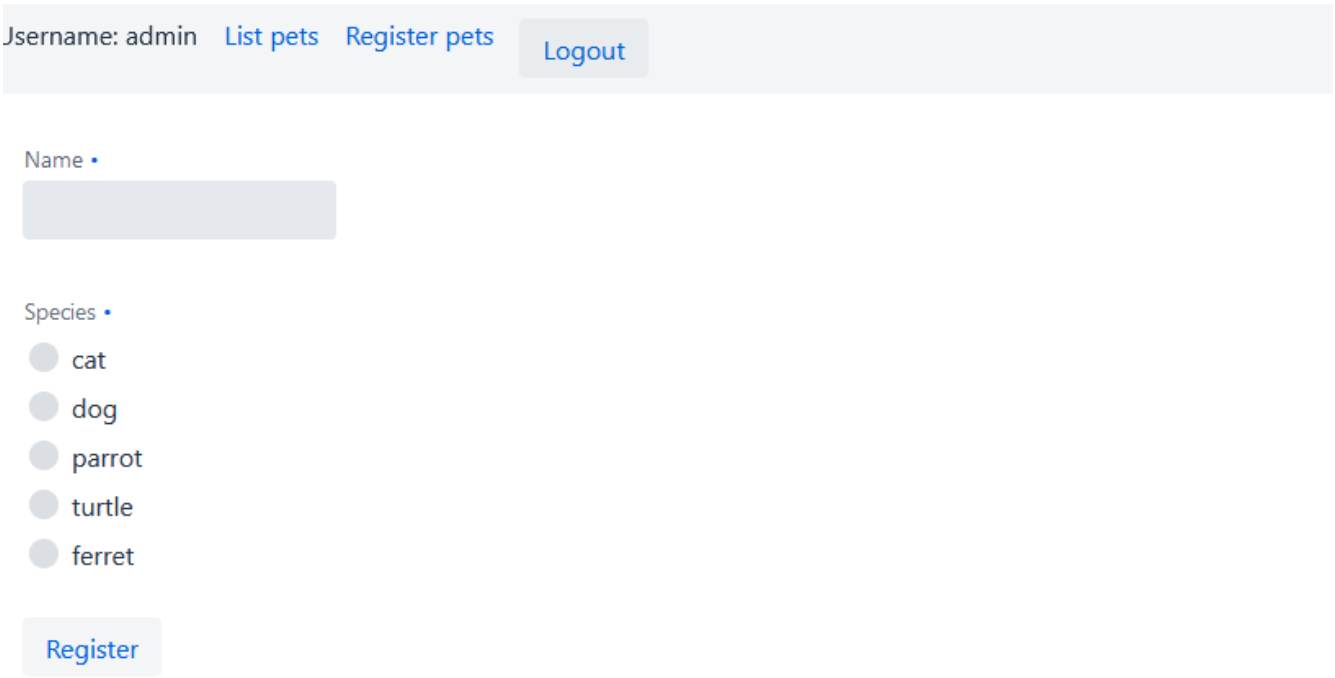
A táblázathoz a *Grid* osztályt használja: https://vaadin.com/docs/latest/components/grid

4 – 2 ListView routing

Állítsa be, hogy a ListView a /list útvonalon legyen elérhető, a MainLayout-ba foglalva.

5. Állat regisztráció

5 – 1 RegisterView



Készítse el a RegisterView tartalmát, hogy a következőképpen nézzen ki:

A faj listát a *PetService.listSpecies* metódusa adja vissza a *PetSpecies* enum alapján dinamikusan. A form és a modell között bindolással teremtsen kapcsolatot. Figyeljen arra, hogy minden mező kitöltése kötelező legyen! A mentés gomb megnyomásakor ellenőrizze az adatok helyességét. Hiba esetén írjon ki egy figyelmeztető üzenetet a felületre!

5 – 2 PetController

Állítsa be, hogy a RegisterView a */register* útvonalon legyen elérhető, a MainLayout-ba foglalva.

6. Képernyők címe

Mindegyik képernyőn állítson be lokalizációnak megfelelő címet!

7. Jogosultság

7 – 1 Global method security

Engedélyezze a global method security-t.

7 – 2 Állat regisztráció

A ‘PetService’ implementációjában érje el, hogy az állat regisztrációhoz fűződő metódusokat csak azok a felhasználók tudják meghívni, akik rendelkeznek ‘REGISTER’ jogosultsággal. A menüben lévő regisztrációs gomb is csak ugyanezzel a jogosultsággal rendelkező felhasználóknak jelenjen meg. A RegisterView csak akkor legyen látogatható, ha rendelkezik a felhasználó ezzel a jogosultsággal.

7 – 3 Állatok szűrése – bónusz feladat

A global method security segítségével érje el, hogy a ‘PetService.listAll()’ metódus implementációja csak azokat az állatokat adja vissza, amikhez az adott felhasználónak joga van. Tehát macska típusú kisállatot csak akkor adjon vissza, ha a felhasználó rendelkezik ‘CAT’ jogosultsággal. A bejelentkezett felhasználó jogosultságainak String-ként való lekérdezésére használja a SecurityService ‘getRolesAsString()’ metódusát. Figyeljen oda az objektumok típusára az összehasonlításkor!